AuthOr: Lower Cost Authenticity-Oriented Garbling of Arbitrary Boolean Circuits

Osman Biçer and Ali Ajorian

University of Basel, Switzerland

Abstract. Authenticity-oriented (previously named as *privacy-free*) garbling schemes of Frederiksen et al. Eurocrypt '15 are designed to satisfy only the authenticity criterion of Bellare et al. ACM CCS '12, and to be more efficient compared to full-fledged garbling schemes. In this work, we improve the state-of-the-art authenticity-oriented version of half gates (HG) garbling of Zahur et al. Crypto '15 by allowing it to be bandwidth-free if any of the input wires of an AND gate is freely settable by the garbler. Our full solution AuthOr then successfully combines the ideas from information-theoretical garbling of Kondi and Patra Crypto '17 and the HG garbling-based scheme that we obtained. AuthOr has a lower communication cost (i.e. garbled circuit or GC size) than HG garbling without any further security assumption. Theoretically, AuthOr's GC size reduction over HG garbling lies in the range between 0 to 100%, and the exact improvement depends on the circuit structure. We have implemented our scheme and conducted tests on various circuits that were constructed by independent researchers. Our experimental results show that in practice, the GC size gain may be up to roughly 98%.

Keywords: Garbled circuits · Privacy-free garbling · Verifiable computing · Zero-knowledge proofs.

1 Introduction

Garbled circuits. Introduced by Andrew Yao [35], garbled circuits are an essential field of study in cryptography with applications to secure two-party computation [36,25,26,28], secure multi-party computation [27,34], identity-based encryption [12], verifiable oursourced computing [15,30], and zero-knowledge protocols [19,13,10,24,14]. In its generic form, a garbled circuit (GC) protocol is a two-party protocol with a garbler party and an evaluator party, such that the former garbles a boolean circuit f known to both parties and the latter evaluates it on encoded inputs to obtain encoded outputs. Classically, the garbling procedure involves assigning two ℓ-bit keys[1] W_i^0 and W_i^1 to each wire i for truth values 0 and 1 to generate the garbled circuit $\hat{F}$. The evaluator then obtains $\hat{F}$ and each key W_i for the truth value w_i of the circuit input wire i. The evaluator proceeds in topological order gate-by-gate to obtain the key W_i for each circuit

[1] Throughout this work, $\ell \in O(\lambda)$ is the key size based on the security parameter λ.

output wire i by using $\hat{F}$. [7] formalized the garbled circuits as a cryptographic primitive with three security notions, privacy, obliviousness, and *authenticity*. While privacy and obliviousness are concerned with keeping as secret the wires' truth values from the evaluator, authenticity prevents the evaluator from forging the encodings of "fake" outputs that are not obtained by honestly evaluating the garbled circuit. The efficiency of a garbled circuit is often determined by three metrics, composed of the computation overheads to the garbler and the evaluator, and the size of the garbled circuit. The latter is considered the main bottleneck [37] in many applications as it directly affects the bandwidth overhead.

Authenticity-oriented (privacy-free) garbling [2] [13] showed that authenticity-oriented garbling schemes without the privacy and the obliviousness properties can be significantly more efficient than fully-fledged schemes. Their idea is that when the evaluator knows the truth values w_a and w_b, it can use them in the evaluation algorithm, reducing the overhead. These schemes are useful for verifiable outsourced computation [15,30] and for efficient zero-knowledge proof protocols [19,13,10,24,14]. The latter application requires an additional verifiability property to ensure that a verifier cannot maliciously construct a garbled circuit whose output keys may reveal the prover's input bits, which correspond to the prover's witness.

There is a quest for achieving an optimal authenticity-oriented garbling scheme. Previous works include the authenticity-oriented schemes by [13], i.e., GRR2 (compatible with FreeXOR), GRR1, and GRR1 with FleXOR. The most relevant techniques to our work are the authenticity-oriented version of half gates (HG) garbling [37] and the information-theoretical (IT) garbling [24]. If the circuit is a boolean formula, i.e., it consists of only fan-out 1 gates, IT garbling is optimal with a garbled circuit of size zero and no cryptographic operations. Although boolean formulas are already an important class of circuits with zero-knowledge proof of satisfiability applications [16], this is still quite restrictive. According to the Shannon bound [33,5], the great majority of circuits with n input bits have size $\Theta(2^n/n)$, whereas a boolean formula with n input bits can have a size $O(n)$. For fairness, we converted an example generic circuit into a boolean formula (see Appendix A), which resulted in a circuit with a size exponential in the size of the original circuit, implying that both garbling and evaluating have exponential complexity [3]. To the best of our knowledge, before this work, HG garbling has remained the most efficient authenticity-oriented garbling scheme applicable to all boolean circuits.

Our contributions. We provide a novel authenticity-oriented garbling scheme that is FreeXOR compatible and has lower AND gate costs than HG garbling. Our technique benefits from the incorporation of IT garbling and a more effi-

[2] We use "authenticity-oriented" instead of the previous naming "privacy-free" to focus on what these protocols achieve instead of what they do not.

[3] The authors also acknowledge this as "not generally efficient for circuits that are not largely formulaic" in [24].

cient version of HG garbling that we also propose here. More concretely, here we achieve the following:

1. In Section 3, along with the previously developed garbling schemes that we use in our final solution, we propose an HG-based AND gate garbling scheme that is FreeXOR compatible. This scheme allows the garbler to garble an AND gate with 0 *ct* if one of the gate input wires is not already set by another gate that comes before it in topological order.
2. In Section 4, we combine the scheme that we mentioned in the previous step with IT garbling to obtain our final authenticity-oriented garbling scheme AuthOr. The garbling Gb algorithm of Author detects based on which gate garbling suits better for each gate and benefits from IT garbling and our proposal from the previous step as much as possible, while the evaluation algorithm Ev detects for each gate which gate garbling is used and proceeds accordingly. Theoretically, for any circuit, compared to HG garbling AuthOr achieves: (i) The same computational security guarantee based on circular correlation robustness (CCR) of the hash function, (ii) 0 to 100% reduced garbled circuit size, and (iii) 0 to 100% less hash function computations for both garbler and evaluator. The exact improvement depends on the circuit.
3. In Section 6, we provide our implementation results for both our scheme and HG garbling by testing those schemes on the boolean circuits available on [4,20]. While for the cryptographic circuits our efficiency improvement can be considered marginal, for other ones we achieve up to 98.44% GC size reduction without blowing up the computation costs.
4. In Appendix 5, we show the security of our scheme based on the security (i.e., satisfying authenticity of [7] and verifiability of [19]) of HG garbling which in turn is based on the circular correlation robustness of the hash scheme used [37].

Comparison with the state-of-the-art. Table 1 compares our scheme with the mentioned authenticity-oriented garbling schemes, the symmetric-key based full-fledged fast garbling of [18], and three halves of [32]. Circular correlation robustness (CCR) [11] is a standard assumption in FreeXOR and FleXOR compatible schemes, while the security of the non-compatible ones can be directly based on its pseudo-randomness.

Comparison to other proposals. We would like to mention some other approaches than symmetric-key-based ones for fairness. Gate-ID-based garbling [21] scheme uses trapdoor permutations (i.e., public-key operations) to garble a circuit, resulting in a garbled circuit with 4 bits per gate and an additional 2 *ct*s for AND gates that share input wires with other gates. While in terms of the garbled circuit size $[4g, 4g + (g-1) \cdot 2ct]$, [21] improves efficiency compared to previous schemes, it relies on public-key primitives. Other approaches include succinct garbling schemes [9,3] and reusable garbling schemes [17,8,1]. However, they induce heavy computational costs and are based on strong assumptions, e.g., the existence of indistinguishability obfuscation / functional encryption / fully homomorphic encryption, the learning with errors (LWE) assumption, or assumptions on multilinear maps. While one recent work [29] based on the LWE

Technique	Gb cost	Ev cost	GC size (bits)	Assump.
GRR2+FreeXOR	$3g_A s$	$g_A s$	$2g_A \ell$	CCR
GRR1	$3g_A s$	$g_A s$	$g\ell$	PRF
GRR1+FleXOR	$3g_A s$	$g_A s$	$[g_A \ell, (g_A + 2g_X)\ell]$	CCR
Fast garbling	$(4g_A + 3g_X)s$	$(2g_A + 1.5g_X)s$	$(2g_A + g_X)\ell$	PRF
Three halves	$[3g_A s, 6g_A s]$	$[1.5g_A s, 3g_A s]$	$1.5g_A \ell + 5g$	CCR
IT	$[\Omega(g), exp(g)]$	$[\Omega(g), exp(g)]$	0	None
HG	$2g_A s$	$g_A s$	$g_A \ell$	CCR
AuthOr	$\mathbf{[0, 2g_A s]}$	$\mathbf{[0, g_A s]}$	$\mathbf{[0, g_A \ell]}$	**CCR**

Table 1: Comparison of state-of-the-art garbling schemes with authenticity, evaluating garbling (Gb), evaluation (Ev), GC size, and security assumptions (CCR=circular correlation robustness and PRF=pseudo-random function). Circuit costs are parameterized by the total number g of gates, the total number g_A of AND gates, the total number g_X of XOR gates, symmetric-key operation cost s, and ciphertext size $\ell \in O(\lambda)$. Ranges reflect variations across circuit structures for f. All schemes include additional $\Theta(g)$ computation overheads from basic operations.

assumption has a very low garbled size (g bits), its concrete computation costs are heavy with a lot of homomorphic operations per gate.

2 Background on Circuits and Garbling

Boolean circuits and notation. A boolean circuit f is a directed acyclic graph with gates, circuit inputs, and circuit outputs as vertices, and wires as edges. Throughout this work, we use the following notation. $i.\mathsf{GateInputs}$ denotes the input wire(s) of the gate i. $f.\mathsf{XORGates}$ denotes the XOR gates in the circuit f. $i.\mathsf{Type}$ denotes the type variable associated with the wire i. Each gate i is identified by the index of their output wire i.

Garbled circuits and notation. Throughout this work,

- $a \twoheadleftarrow B$ denotes that a is picked uniformly at random from the set B,
- $a \leftarrow B$ denotes that a is set as an execution of the algorithm B,
- $\ell \in O(\lambda)$ is the key length chosen based on the security parameter and symmetric-key primitive,
- λ denotes the security parameter. $f.\mathsf{Inputs}$ denotes the input wires of the circuit f,
- $f.\mathsf{Outputs}$ denotes the output wires of the circuit f,
- $\hat{a}$ denotes a straightforward collection of the values produced as a_i,
- $\hat{F}.\mathsf{Next}$ denotes the iterator over the garbled circuit $\hat{F}$.

This work follows the garbling schemes abstraction introduced by [7]. A garbling scheme is composed of the following algorithms:

– Gb: Takes as input 1^λ and a boolean circuit f, and outputs $\hat{F}$, $\hat{e}$, and $\hat{d}$, which denote the garbled circuit, encoding information, and decoding information, respectively.
– Ev: Takes as input f, $\hat{F}$, and $\hat{X}$, and outputs the garbled output $\hat{Y}$.
– En: Takes as input $\hat{e}$ and the plaintext input bit string $\hat{x}$ to f, and outputs garbled input $\hat{X}$.
– De: Takes as input $\hat{d}$ and $\hat{Y}$, and outputs the plaintext output bit string $\hat{y}$.
– Ve: Takes as input $\hat{e}$, $\hat{F}$, and f and outputs a single bit b (1 for verified).

Definition 1 (Correctness). *A garbling scheme is correct if for every circuit* $f : \{0,1\}^n \rightarrow \{0,1\}^m$ *and its input string* $\hat{x} \in \{0,1\}^n$, $f(x) = \mathsf{De}\Big(\hat{d}, \mathsf{Ev}\big(f, \hat{F}, \mathsf{En}(\hat{e}, \hat{x})\big)\Big)$ *holds for each execution* $(\hat{F}, \hat{e}, \hat{d}) \leftarrow \mathsf{Gb}(1^\lambda, f)$.

As we are interested in authenticity-oriented garbling schemes, we provide the authenticity definition of [7] below. The definition briefly ensures no PPT adversary can come up with incorrect output wire keys that are decodable by De.

Definition 2 (Authenticity). *A garbling scheme is authentic if for every PPT adversary* $\mathcal{A}$*, every circuit* $f : \{0,1\}^n \rightarrow \{0,1\}^m$ *and its input string* $\hat{x} \in \{0,1\}^n$*, there exist a negligible function* n *such that:*

$$Pr[(\hat{Y} \neq \mathsf{Ev}(f, \hat{F}, \hat{X})) \wedge (\mathsf{De}(\hat{d}, \hat{Y}) \neq \perp) : (\hat{F}, \hat{e}, \hat{d}) \leftarrow \mathsf{Gb}(1^\lambda, f), \hat{X} \leftarrow \mathsf{En}(\hat{e}, \hat{x}),$$
$$\hat{Y} \leftarrow \mathcal{A}(\hat{F}, \hat{X})] \leq \mathsf{n}(\lambda).$$

Additionally, we require the authenticity-oriented garbling scheme to have the verifiability of [19], so that the obtained garbling scheme can easily be plugged into their given zero-knowledge proof (ZKP) of knowledge protocol. In this ZKP scheme, the verifier plays the garbler role during the garbled circuit subprotocol, yet security against a malicious verifier/garbler is still achieved if the verifiability of the garbled circuit is ensured. We note that the definition below is a slightly modified version of the one given in [19] for arbitrary output lengths by giving y as input to Ext.

Definition 3 (Verifiability). *A garbling scheme is verifiable if for every PPT adversary* $\mathcal{A}$*, every circuit* $f : \{0,1\}^n \rightarrow \{0,1\}^m$*, there exist an expected polynomial time algorithm* Ext *and a negligible function* n *such that:*

1. $Pr[\mathsf{Ev}(f, \hat{F}, \hat{X}_0) \neq \mathsf{Ev}(f, \hat{F}, \hat{X}_1) \wedge \mathsf{Ve}(\hat{e}, \hat{F}, f) = 1 : (\hat{F}, \hat{e}) \leftarrow \mathcal{A}(1^\lambda, f), \hat{X}_0 \leftarrow \mathsf{En}(\hat{e}, \hat{x}_0), \hat{X}_1 \leftarrow \mathsf{En}(\hat{e}, \hat{x}_1)] \leq \mathsf{n}(\lambda)$ *for input strings* $\hat{x}_0, \hat{x}_1 \in \{0,1\}^n$ *s.t.* $f(\hat{x}_0) = f(\hat{x}_1)$*, and*
2. $Pr[\mathsf{Ext}(\hat{F}, \hat{e}, y) \neq \mathsf{Ev}(f, \hat{F}, \hat{X}) \wedge \mathsf{Ve}(\hat{e}, \hat{F}, f) = 1 : (\hat{F}, \hat{e}) \leftarrow \mathcal{A}(1^\lambda, f), \hat{X} \leftarrow \mathsf{En}(\hat{e}, \hat{x})] \leq \mathsf{n}(\lambda)$ *for all input strings* $\hat{x} \in \{0,1\}^n$ *s.t.* $f(\hat{x}) = \hat{y}$.

procedure $\mathsf{GbHG2}(i, W_a^0, W_b^0, \Delta)$: $W_i^0 \leftarrow H(i, W_a^0)$ $F_i \leftarrow H(i, W_a^0) \oplus$ $H(i, W_a^0 \oplus \Delta) \oplus W_b^0$ **return** (F_i, W_i^0)	**procedure** $\mathsf{GbHG1}(i, W_a^0, \Delta)$: $W_i^0 \leftarrow H(i, W_a^0)$ $W_b^0 \leftarrow H(i, W_a^0) \oplus$ $H(i, W_a^0 \oplus \Delta)$ **return** (W_b^0, W_i^0)	**procedure** $\mathsf{GbHG0}(i, \Delta)$: $W_a^0 \twoheadleftarrow \{0,1\}^\ell$ $W_i^0 \leftarrow H(i, W_a^0)$ $W_b^0 \leftarrow H(i, W_a^0) \oplus H(i, W_a^0 \oplus \Delta)$ **return** (W_a^0, W_b^0, W_i^0)
procedure $\mathsf{GbFreeXOR1}(W_a^0)$: $W_b^0 \twoheadleftarrow \{0,1\}^\ell$ $W_i^0 \leftarrow W_a^0 \oplus W_b^0$ **return** (W_b^0, W_i^0)	**procedure** $\mathsf{GbFreeXOR0}()$: $W_a^0 \twoheadleftarrow \{0,1\}^\ell$ $W_b^0 \twoheadleftarrow \{0,1\}^\ell$ $W_i^0 \leftarrow W_a^0 \oplus W_b^0$ **return** (W_a^0, W_b^0, W_i^0)	**procedure** $\mathsf{GbFreeXOR2}(W_a^0, W_b^0)$: $W_i^0 \leftarrow W_a^0 \oplus W_b^0$ **return** W_i^0
procedure $\mathsf{GbITAND}(W_i^0, W_i^1)$: $W_a^0, W_b^0 \leftarrow W_i^0, W_i^0$ $W_a^1 \twoheadleftarrow \{0,1\}^\ell$ $W_b^1 \leftarrow W_a^1 \oplus W_i^1$ **return** $(W_a^0, W_a^1, W_b^0, W_b^1)$	**procedure** $\mathsf{GbITXOR}(W_i^0, W_i^1)$: $W_a^1 \twoheadleftarrow \{0,1\}^\ell$ $W_b^1 \leftarrow W_a^1 \oplus W_i^0$ $W_a^0 \leftarrow W_b^1 \oplus W_i^1$ $W_b^0 \leftarrow W_a^1 \oplus W_i^1$ **return** $(W_a^0, W_a^1, W_b^0, W_b^1)$	**procedure** $\mathsf{GbBwNOT}(W_i^0, W_i^1)$: $(W_a^0, W_a^1) \leftarrow (W_i^1, W_i^0)$ **return** (W_a^0, W_a^1) **procedure** $\mathsf{GbFwNOT}(W_a^0, \Delta)$: $W_i^0 \leftarrow W_a^0 \oplus \Delta$ **return** W_i^0

Table 2: Gate garbling algorithms that are used in AuthOr.

3 Gate Garbling Algorithms and Our Optimization on Half Gates

In this section, we elaborate on the algorithms that we use in AuthOr, i.e. authenticity-oriented half gates-based HG2, HG1, HG0, and information-theoretic ITAND for garbling AND gates; FreeXOR based FreeXOR2, FreeXOR1, FreeXOR0, and information-theoretic ITXOR for garbling XOR gates; FwNOT and BwNOT for garbling NOT gates. Among the given gate garbling techniques, HG1 and HG0 are our contribution. We highlight that all the given algorithms garble gates in the forward direction in topological order, except for IT garbling algorithms, which garble in the backward direction from outputs to inputs, but we postpone how to determine which algorithm will be used for a given gate to Section 4. Tables 2, 3, and 4 show the formal garbling, evaluation, and verification algorithms, respectively.

FreeXOR (FreeXORz) [23]. The FreeXOR technique [23] requires that the keys for each wire i have the same offset Δ, so $W_i^1 = W_i^0 \oplus \Delta$ always holds. Given the input wire keys W_a^0 and W_b^0, FreeXOR sets the output wire key $W_i^0 \leftarrow W_a^0 \oplus W_b^0$. Depending on the number z of inputs that have been set before garbling the XOR gate, this scheme is called as FreeXORz for $z \in \{0, 1, 2\}$ and they differ by picking the unset wire key uniformly at random. The corresponding garbling algorithms are named as GbFreeXOR2, GbFreeXOR1, and GbFreeXOR0; and the evaluation algorithm for all XOR gates is EvXOR.

Authenticity-Oriented Half Gates Garbling (HGz). Here we provide the HG2 of [37] and our contributions HG1 and HG0 obtained from it.

HG2 **[37].** Authenticity-Oriented version of the half-gates protocol [37] is compatible with FreeXOR, i.e., the keys for each wire w_i have the same offset Δ. Here, we provide a slightly modified version, such that the hash call H takes

procedure $\mathsf{EvHG2}(i, F_i, W_a, W_b, w_a, w_b)$:	**procedure** $\mathsf{EvHG0\&1}(i, W_a, W_b, w_a, w_b)$:
if $w_a = 0$ **then**	**if** $w_a = 0$ **then**
$W_i \leftarrow H(i, W_a)$	$W_i \leftarrow H(i, W_a)$
else	**else**
$W_i \leftarrow H(i, W_a) \oplus W_b \oplus F_i$	$W_i \leftarrow H(i, W_a) \oplus W_b$
return W_i	**return** W_i
procedure $\mathsf{EvXOR}(W_a, W_b, w_a, w_b)$:	**procedure** $\mathsf{EvITAND}(W_a, W_b, w_a, w_b)$:
$W_i \leftarrow W_a \oplus W_b$	**if** $w_a = 0$ **then**
return W_i	$W_i \leftarrow W_a$
procedure $\mathsf{EvNOT}(W_a)$:	**else if** $w_b = 0$ **then**
$W_i \leftarrow W_a$	$W_i \leftarrow W_b$
return W_i	**else**
	$W_i \leftarrow W_a \oplus W_b$
	return W_i

Table 3: Gate evaluation algorithms that are used in AuthOr.

as input the id i of the gate instead of the one in [37] [4]. Given the input wire keys W_a^0, W_b^0, and the offset Δ, the garbling algrorithm $\mathsf{GbHG2}$ for the gate with index i results in 1 *ct* which is set as $F_i \leftarrow H(i, W_a^0) \oplus H(i, W_a^1) \oplus W_b^0$ where $H : \{0,1\}^* \rightarrow \{0,1\}^\ell$ is an hash function with circular correlation robustness [11]. The output wire key for 0 is set as $W_i^0 = H(i, W_a^0)$. As this scheme requires that input keys for both wires should have been already fixed before the garbling, we call this gate garbling as $\mathsf{HG2}$. We note that among all gate garbling schemes that we use, this one is the only one that results in a ciphertext that needs to be sent to the evaluator. If the truth value $w_a = 0$, the evaluation algorithm $\mathsf{EvHG2}$, sets $W_i \leftarrow H(i, W_a)$. Otherwise, $W_i \leftarrow F_i \oplus H(i, W_a) \oplus W_b$.

Our contribution: $\mathsf{HG1}$ and $\mathsf{HG0}$. If the keys of only one input wire was not fixed before the garbling of a gate, then the garbler can choose such that $F_i = 0^\ell$ and it does not need to be sent anymore. More concretely, given the input wire key W_a^0 and the offset Δ, the key of the output wire i is set as $W_i^0 = H(i, W_a^0)$ by $\mathsf{GbHG1}$, the key for the w_b is set as $W_b^0 \leftarrow H(i, W_a^0) \oplus H(i, W_a^1)$. If none of the wires were set before the gate being garbled, one wire key is picked randomly as $W_a^0 \twoheadleftarrow \{0,1\}^\ell$ and the algorithm $\mathsf{GbHG0}$ continues in the same manner as $\mathsf{HG1}$. Given the input keys W_a and W_b, if $w_a = 0$, the evaluation algorithm $\mathsf{EvHG0\&1}$ for both $\mathsf{HG1}$ and $\mathsf{HG0}$ sets $W_i \leftarrow H(i, W_a)$. Otherwise, $W_i \leftarrow H(i, W_a) \oplus W_b$.

Information-Theoretic (IT) Garbling [24]. [24] provides an elegant way of authenticity-oriented garbling AND and XOR gates that does not require costly hash function calls and results in size-zero garbled circuits. However, if during the evaluation of an AND gate $w_a = 0$ and $w_b = 1$, their scheme leaks the keys both W_b^0 and W_b^1 as it sets $W_a^0 = W_b^0$ during garbling. Yet, the authors prove in their work, this leakage is not a problem (as the output wire key is never leaked) as long as there is no fan-out-2 gates in the circuit. Due to this leakage, their scheme does not have a global offset Δ. Nevertheless, the authors still show that XOR gates can be garbled with a local offset "freely". The garbling

[4] In [37], the hash function takes as input a value incremented with each AND gate garbled. This modification does not affect the security at all, as their proof is based on the uniqueness of this value.

procedure VeHG2$(i, W_a^0, W_a^1, W_b^0, W_b^1, F_i)$:	**procedure** VeHG0&1$(i, W_a^0, W_a^1, W_b^0, W_b^1)$:
if $W_a^0 \oplus W_a^1 \neq W_b^0 \oplus W_b^1$	**if** $W_a^0 \oplus W_a^1 \neq W_b^0 \oplus W_b^1$
return 0	**return** 0
$\Delta \leftarrow W_a^0 \oplus W_a^1$	$\Delta \leftarrow W_a^0 \oplus W_a^1$
$K_1 \leftarrow H(i, W_a^0)$	$K_1 \leftarrow H(i, W_a^0)$
$K_2 \leftarrow W_b^0 \oplus H(i, W_a^0 \oplus \Delta) \oplus F_i$	$K_2 \leftarrow W_b^0 \oplus H(i, W_a^0 \oplus \Delta)$
if $K_1 \neq K_2$	**if** $K_1 \neq K_2$
return 0	**return** 0
return $(K_1, K_1 \oplus \Delta)$	**return** $(K_1, K_1 \oplus \Delta)$
procedure VeXOR$(W_a^0, W_a^1, W_b^0, W_b^1)$:	**procedure** VeNOT(W_a^0, W_a^1):
if $W_a^0 \oplus W_a^1 \neq W_b^0 \oplus W_b^1$	**return** (W_a^1, W_a^0)
return 0	**procedure** VeITAND$(W_a^0, W_a^1, W_b^0, W_b^1)$:
return $(W_a^0 \oplus W_b^0, W_a^0 \oplus W_b^1)$	**if** $W_a^0 \neq W_b^0$
	return 0
	return $(W_a^0, W_a^1 \oplus W_b^1)$

Table 4: Gate verification algorithms that are used in AuthOr.

is "backward"' that is visiting gates in the inverse topological order, so that when a gate is being garbled, either its output keys are already set or the garbler picks them randomly as $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ and $W_i^1 \twoheadleftarrow \{0,1\}^\ell$).

ITAND. Given the output keys W_i^0 and W_i^1, the garbling GbITAND sets the keys for the truth value zero for both input wires as $W_a^0 \leftarrow W_i^0$ and $W_b^0 \leftarrow W_i^0$. Next, it picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$ and computes $W_b^1 \leftarrow W_a^1 \oplus W_i^1$. During the evaluation, EvITAND sets $W_i \leftarrow W_a$ if $w_a = 0$, $W_i \leftarrow W_b$ if $w_a = 1$ and $w_b = 0$, and $W_i \leftarrow W_a \oplus W_b$ otherwise.

ITXOR. Given the output keys W_i^0 and W_i^1, the garbling GbITXOR sets the local offset for the XOR gate being garbled as follows. It picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$ and computes $W_b^1 \leftarrow W_a^1 \oplus W_i^0$, $W_a^0 \leftarrow W_b^1 \oplus W_i^1$, and $W_b^0 \leftarrow W_a^1 \oplus W_i^1$. During the evaluation, EvXOR sets $W_i \leftarrow W_a \oplus W_b$ using.

Garbling NOT Gates [22,24]. In both forward and backward directions, NOT gates can be freely garbled by just swapping the association of keys for 0 and 1 as in [22,24]. That is, given a NOT gate with input wire a and output wire i, $W_a^0 = W_i^1$ and $W_a^1 = W_i^0$ based on in which direction this is done, we name the algorithms as GbFwNOT for forward and GbBwNOT for backward. The evaluation algorithm EvNOT just copies the key for the input to the output in the forward direction, i.e., the key does not change while the truth value changes.

Verification Algorithms. Figure 2 also shows the verification algorithms for the provided gate garbling algorithms. Given both keys for each input wire, they check whether a gate is garbled correctly. If so, they return 1 and the keys for the output wire; otherwise, they return 0. We have obtained VeITAND from [24] as the verification algorithm for the IT garbled AND gates, which checks that the output key for 0 that could be found in all cases would be equal. The verification algorithm VeXOR for XOR gates executes based on whether both input wires have the same offset. We constructed the half gates AND verification algorithms VeHG2 and VeHG20&1 based on the check that the output key for 0 that could be found in all cases would be equal or not similar to VeITAND. We

highlight that VeXOR, VeHG2, and VeHG20&1 also check whether both input wires have the same offset.

4 Our Authenticity-Oriented Garbling Protocol

We provide our AuthOr scheme in Table 5 that makes use of the gate garbling algorithms in Section 3. We assume both input wires for each AND or XOR gate are distinct, as otherwise, our scheme is not immune to the mentioned attack in [31] [5].

Initialization of our Gb algorithm. The Gb algorihtm starts by choosing a global offset Δ. The type of each wire is initially set by Gb as TypeS if that wire is input to only at most a *single* gate. Otherwise, it is initially set as TypeM. Here, we do not propose a method for this labeling, as there are simple ways to do this in $O(g)$ that are derived from generic topological sorting algorithms.

Forward and backward phases of Gb. In our solution, we attempt to use IT garbling in as many gates as possible. To solve the wire key leakage issue of IT garbling mentioned in Section 3, our garbling algorithm Gb has two phases: a *forward* phase i.e., garbling in the topological order with GbFwNOT, GbHGz, and GbFreeXORz garbling with a global offset Δ; and a *backward* phase, i.e., garbling in the inverse topological order with GbBwNOT, ITAND, and ITXOR.

Determination of the gate garbling algorithms. The gate garbling algorithm chosen for a gate is based on the types of its input wires. During the forward phase, only the gates that would result in a problem for IT garbling (if they were left to the backward phase) are garbled, and the offsets of input and output wires of those gates are set as Δ. The gates with a TypeM input wire cannot be left to the backwards phase, as the IT garbling results in the leakage of both keys for a wire, which is known to be a problem if that wire is input to more than one gate. Also, during the forward phase of Gb, when the keys of a wire is *determined* with offset Δ, we cannot leave that wire to the backward phase, as the leakage of both keys means leakage of Δ and the compromise of the security of all wires with the offset Δ. Hence, during the forward phase Gb replaces such wire types with TypeF and also garbles each gate that those wires are input. Gb leaves for the backward phase only the gates whose both inputs are TypeS input wires, when the time of garbling that gate comes. We highlight that the keys for these wires cannot be set later during the forward phase, due to the following theorem:

Theorem 1. *Once a wire that is input to a gate i is labeled as* TypeS *by the* Gb *algorithm, then during the forward phase, its label may be changed into* TypeF *only when a gate $j \leq i$ is being garbled.*

Proof. Wires are initially labeled as either TypeS or TypeM, and the algorithm never reassigns a wire to TypeS later. Similarly, a wire labeled TypeS cannot become TypeM, since TypeM is only assigned at initialization.

[5] If this is not the case, we refer to their work for the possible countermeasures.

procedure $\mathsf{Gb}(1^\lambda, f)$:
  $\Delta \twoheadleftarrow \{0,1\}^\ell$ s.t. $\ell \in O(\lambda)$
  label each wire as TypeM or TypeS
  $\{Forward\ phase\}$
  for each gate $i \in f$ $\{in\ topo.\ order\}$ **do**
    if $i \in f$.NOTGates **then**
      $a \leftarrow i$.GateInputs
      if a.Type = TypeM **then**
        $W_a^0 \twoheadleftarrow \{0,1\}^\ell$
      if a.Type $\neq$ TypeS **then**
        $W_i^0 \leftarrow \mathsf{GbFwNOT}(W_a^0)$

    else if $i \in f$.XORGates **then**
      $\{a,b\} \leftarrow i$.GateInputs
      if a.Type = TypeF **and** b.Type = TypeF **then**
        $W_i^0 \leftarrow \mathsf{GbFreeXOR2}(W_a^0, W_b^0)$

      else if $\exists p \in \{a,b\}$ $s.t.$ p.Type = TypeF **then**
        $(W^0_{\{a,b\}\setminus p}, W_i^0) \leftarrow \mathsf{GbFreeXOR1}(W_p^0)$

      else if $\exists p \in \{a,b\}$ $s.t.$ p.Type = TypeM **then**
        $(W_a^0, W_b^0, W_i^0) \leftarrow \mathsf{GbFreeXOR0}()$

    else
      $\{a,b\} \leftarrow i$.GateInputs
      if a.Type = TypeF **and** b.Type = TypeF **then**
        $(\hat{F}.\mathsf{Next}, W_i^0) \leftarrow \mathsf{GbHG2}(i, W_a^0, W_b^0, \Delta)$

      else if $\exists p \in \{a,b\}$ $s.t.$ p.Type = TypeF **then**
        $(W^0_{\{a,b\}\setminus p}, W_i^0) \leftarrow \mathsf{GbHG1}(i, W_p^0, \Delta)$

      else if $\exists p \in \{a,b\}$ $s.t.$ p.Type = TypeM **then**
        $(W_a^0, W_b^0, W_i^0) \leftarrow \mathsf{GbHG0}(i, \Delta)$

    if gate i has TypeM or TypeF input wire **then**
      for each $p \in \{a,b,i\}$ s.t. p.Type $\neq$ TypeF **do**
        $W_p^1 \leftarrow W_p^0 \oplus \Delta$
        label p as TypeF

  $\{Backward\ phase\}$
  for each gate $i \in f$ $\{in\ inv.\ topo.\ order\}$ **do**
    if i.Type = TypeS **then**
      $(W_i^0, W_i^1) \twoheadleftarrow \{0,1\}^\ell \times \{0,1\}^\ell$

    if $i \in f$.NOTGates **then**
      $a \leftarrow i$.GateInputs
      if a.Type = TypeS
        $(W_a^0, W_a^1) \leftarrow \mathsf{GbBwNOT}(W_i^0, W_i^1)$

    if $i \in f$.XORGates **then**
      $(a,b) \leftarrow i$.GateInputs
      if a.Type = TypeS **then**
        $(W_a^0, W_a^1, W_b^0, W_b^1) \leftarrow \mathsf{GbITXOR}(W_i^0, W_i^1)$

    else
      $(a,b) \leftarrow \mathsf{GateInputs}(f, i)$
      if a.Type = TypeS **then**
        $(W_a^0, W_a^1, W_b^0, W_b^1) \leftarrow \mathsf{GbITAND}(W_i^0, W_i^1)$

    label all TypeS input wires of gate i as TypeB

  $\{finalizing\ and\ return\}$
  for $i \in f$.Inputs **do**
    $e_i \leftarrow (W_i^0, W_i^1)$
  for $i \in f$.Outputs **do**
    $d_i \leftarrow (H(W_i^0, i), H(W_i^1, i))$
  return $(\hat{F}, \hat{e}, \hat{d})$

procedure $\mathsf{Ev}(f, \hat{F}, \hat{X})$:
  label each wire as TypeM or TypeS
  for each gate $i \in f$ $\{in\ topo.\ order\}$ **do**
    if $i \in f$.NOTGates **then**
      $a \leftarrow i$.GateInputs
      $W_i \leftarrow \mathsf{EvNOT}(W_a)$

    else if $i \in f$.XORGates **then**
      $\{a,b\} \leftarrow i$.GateInputs
      $W_i \leftarrow \mathsf{EvXOR}(W_a, W_b)$

    else
      $\{a,b\} \leftarrow i$.GateInputs
      if a.Type = TypeF **and** b.Type = TypeF **then**
        $W_i \leftarrow \mathsf{EvHG2}(i, \hat{F}.\mathsf{Next}, W_a, W_b, w_a, w_b)$
      else if $\exists p \in \{a,b\}$ $s.t.$ p.Type = TypeF **then**
        $W_i \leftarrow \mathsf{EvHG0\&1}(i, W_p, W_{\{a,b\}\setminus p}, w_p, w_{\{a,b\}\setminus p})$
      else if $\exists p \in \{a,b\}$ $s.t.$ p.Type = TypeM **then**
        $W_i \leftarrow \mathsf{EvHG0\&1}(i, W_a, W_b, w_a, w_b)$
      else
        $W_i \leftarrow \mathsf{EvITAND}(W_a, W_b, w_a, w_b)$

    if gate i has TypeM or TypeF input wire **then**
      for each $p \in \{a,b,i\}$ s.t. .Type $\neq$ TypeF **do**
        label p as TypeF

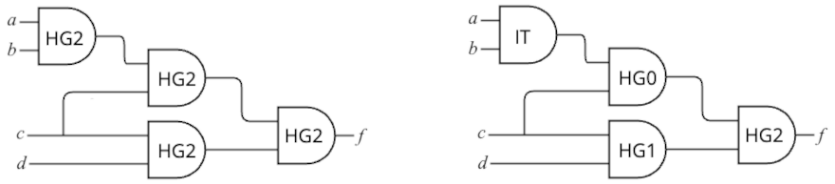
  for $i \in \hat{f}$.Outputs **do**
    $Y_i \leftarrow W_i$
  return $\hat{Y}$

procedure $\mathsf{En}(\hat{e}, \hat{x})$:
  parse $(e_i^0, e_i^1) \leftarrow e_i$
  $X_i \leftarrow e_i^{x_i}$
  return $\hat{X}$

procedure $\mathsf{De}(\hat{d}, \hat{Y})$:
  for $d_i \in \hat{d}$ **do**
    parse $(h_0, h_1) \leftarrow d_i$
    parse $(h_0, h_1) \leftarrow d_i$
    if $H(Y_i, i) = h_0$ **then** $y_i \leftarrow 0$
    else if $H(Y_i, i) = h_1$ **then** $y_i \leftarrow 1$
    else return $\perp$
  return $\hat{y}$

procedure $\mathsf{Ve}(\hat{e}, \hat{F}, f)$:
  find which garbling is used in each gate (as in Ev)
  for each gate $i \in f$ $\{in\ topo.\ order\}$ **do**
    run the related gate verification algorithm
    if it the gate verification returns 0 **then**
      return 0
    else
      assign what verification returned as (W_i^0, W_i^1)
  return 1

Table 5: Our authenticity-oriented garbling scheme AuthOr.



(a) HG Garbled (b) AuthOr Garbled

Fig. 1: The impact of using HG0, HG1 and ITAND on the garbled circuit. The circuits are in topological order, left to right .

A TypeS wire becomes TypeF only when it is involved in the garbling of a gate, either as an input or as an output. If a is input to multiple gates, it cannot be labeled TypeS initially. If it is an output of some gate $j > i$, this violates the topological ordering (since a is also an input to gate i), and so such a gate cannot exist. Hence, only gates $j \leq i$ can change the label of a from TypeS to TypeF. □

Somewhat surprisingly, even the gates whose input wires are TypeS but output wires are converted to TypeF by garbling another gate afterward can be left to the backward phase, as the offset of an output wire is perfectly hidden in IT garbling. We note that an AND gate that is garbled during the forward phase may still benefit from the efficiency improvements due to our HG0 or HG1 if at least one of its inputs is not TypeF before garbling this gate. Also, the garbled circuit $\hat{F}$ is determined during this phase, leaving only the complete determination of input and output keys to the backward phase. During the backward phase, all the gates that are left from the forward case are garbled via IT garbling. Note that these are the gates with both input wires are TypeS so their keys can be set by GbITAND and GbITXOR without any problem. At the end, all wires become either TypeF (each with global offset Δ) or TypeB (each with arbitrary offset). We highlight that if the given circuit f is a boolean formula, all the gates are garbled in the backward phase, and our scheme is equivalent to IT garbling.

Figure 1 compares the number of HG2 runs employed in garbling a generic circuit for both the HG garbling approach [37] and AuthOr. In the HG approach, all 4 gates of the given circuit are garbled using HG2, resulting in 4 cts. In contrast, the AuthOr garbling algorithm applies only one HG2 execution, leading to the generation of a single ct.

Our Ev, En, De, and Ve algorihtms. The evaluation algorithm Ev just runs in the topological order and applies the related evaluation algorithms given in Table 3. What this algorithm needs is only to have information on which gate garbling scheme is used for each gate, which is determined using the same method Gb uses. All the gates garbled in the forward phase of Gb phase can be evaluated with the corresponding evaluation algorithm. The encoding algorithm En is just collecting the keys for the circuit input wires as in [37]. The decoding

algorithm De collects the hash outputs of the keys for the circuit output wires as in [37], where the hash algorithm ensures that even with the decoding data, the evaluator is prevented from finding both truth value keys for the output wire. The verification algorithm Ve finds the gate garbling algorithm that is used in each gate and then, in topological order, calls the gate verification algorithms in Figure 2. We highlight that there is no need for checking the same global offset Δ is used in each gate. Instead, we check whether the incoming wires of a gate have the same offset, which is sufficient to ensure that any set of forward-garbled connected[6] gates have wires with the same local offset. Two such unconnected forward-garbled gate sets could have separate local offsets (although for simplicity, our algorithm does not allow it) and still pass the verification without sacrificing verifiability.

Theorem 2. *The AuthOr garbling scheme is an authenticity-oriented garbling scheme with correctness, authenticity, and verification properties if the hash function H has circular correlation robustness (CCR).*

We show the security of our scheme in Section 5.

5 Security of AuthOr

In what follows show the authenticity and verification properties AuthOr, as correctness simply follows from the correctness of the gate garbling schemes and the use of the same wire labelling for Gb and Ev.

Theorem 3. *The AuthOr garbling scheme satisfies authenticity if the hash function H has circular correlation robustness (CCR).*

Proof. The proof that we provide here depends on the already proven security of the previous schemes, forward garbled (FreeXOR and HG) and backward garbled (IT), and the security of their intersection. During the forward phase of Gb, if the keys for a wire i have been determined (turning that wire into TypeF), then the keys for other input and output wires of a gate that takes i as input are also determined (set as TypeF). We deduce that at the end of a Gb execution, the TypeF wires look like "peninsula"(s) leaning to the output of the circuit in a sea of TypeB wires, which is visualized in Figure 2.

TheTypeB wire sea is already proven to leak no information (including XOR, AND, and NOT gates) about the other output wire keys than the expected ones to be obtained during Ev by [24]. Regarding the TypeF wire peninsulas, they are a simple extension HG* of authenticity-oriented HG garbling scheme of [37] that we define here. HG* garbles only in the forward direction, including FwNOT, which is already shown to have authenticity by [22], HG2 of [37], our proposal HG0 and HG1, and XOR garbling algorithms FreeXOR0, FreeXOR1, and FreeXOR2. In what follows, we show this scheme itself has authenticity:

[6] We slightly abuse the terminology as two gates can be connected via sharing any wires with each other or they are both connected to another gate.

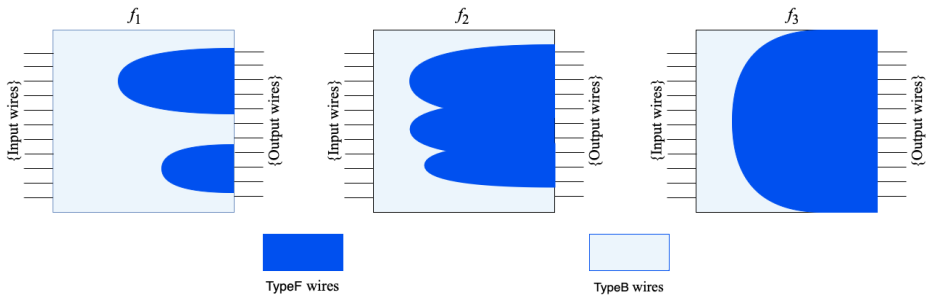


Fig. 2: Example formations of TypeF wires and TypeB wires upon Ga executions on example f_1, f_2, and f_3 (left to right in topological order).

Lemma 1. *HG* garbling scheme (with* HGz, FreeXORz, *and* FwNOT *for* $z \in \{0,1,2\}$*) has authenticity if the authenticity-oriented HG garbling scheme of [37] (with* HG2, FreeXORz, *and* GbFwNOT, *without* HG0, HG1*) has authenticity.*

Proof. We construct a PPT adversary $\mathcal{B}$ that can break the authenticity of HG garbling (played with a challenger $\mathcal{C}$), by using as a subroutine a PPT adversary $\mathcal{A}$ that breaks the authenticity of HG* garbling scheme.

1. $\mathcal{B}$ starts execution of $\mathcal{A}$. $\mathcal{A}$ picks a function f and an input string $\hat{x}$ and outputs them to $\mathcal{B}$, which in turn, outputs them to $\mathcal{C}$.
2. $\mathcal{B}$ obtains the challenge garbled circuit $\hat{F}$ and input $\hat{X}$ from $\mathcal{C}$.
3. $\mathcal{B}$ executes the evaluation algorithm of HG garbling to obtain the key W_i of each wire i.
4. $\mathcal{B}$ finds the set $\hat{G}$ of AND gates that would be garbled with HG0 and HG1 in a normal execution of the garbling algorithm of HG*. Here, we assume the ones that would be garbled with HG1 would have the left wire a as TypeF [7].
5. $\mathcal{B}$ executes the following updates on $\hat{F}$ and $\hat{X}$.

   for each gate $i \in f$ {*in topo. order*} **do**
   if $i \in$G **then**
   $\{a,b\} \leftarrow i$.GateInputs, Set $W_b^* \leftarrow W_b \oplus F_i$, Delete F_i from $\hat{F}$
   Set each $W_b \in \hat{X}$ as the freshly generated key W_b^*

   The updated $\hat{F}$ and $\hat{X}$ are information-theoretically indistinguishable from what could be obtained in an honest execution of HG* (explained below).
6. $\mathcal{B}$ gives $\hat{F}$ and $\hat{X}$ to $\mathcal{A}$.
7. $\mathcal{A}$ gives the output $\hat{Y}$ to $\mathcal{B}$, which, in turn gives it to $\mathcal{C}$.

In an honest execution of HG*, for a given W_a^0, GbHG1 sets W_b^0 as $H(i, W_a^0) \oplus H(i, W_a^1)$. Here, $\mathcal{B}$ additionally picks F_i uniformly random and sets W_b^0 as $H(i, W_a^0) \oplus H(i, W_a^1) \oplus F_i$. On the other hand, the HG garbling picks W_b^0 uniformly random, and then uses GbHG2 to set F_i as $H(i, W_a^0) \oplus H(i, W_a^1) \oplus W_b^0$.

[7] For other circuits, it is trivial to find an equivalent that fits into this criterion as AND gates are symmetric

Obtained W_b^0 and F_i are information-theoretically indistinguishable from each other. The probability that $\mathcal{A}$ comes up with an authentic $\hat{Y}$ that is not equal to what could be obtained in honest execution of the evaluation algorithm on the given $\hat{F}$ and $\hat{X}$ is exactly the same for $\mathcal{B}$ also does so. Hence, if it is non-negligible, $\mathcal{B}$ breaks the authenticity of HG with non-negligible probability. □

So far, we have shown that the peninsulas of TypeF wires have authenticity given that the garbling scheme of [37] has authenticity (which is based on CCR of H in their work), while [24] already shows the surrounding sea of TypeB wires has information-theoretic authenticity. To conclude, we now show that the IT garbling scheme of [24] could be altered so that an arbitrary number of circuit output wires can have the same offset without breaking the information-theoretic authenticity. We name the altered scheme as IT* garbling, which exactly has the garbling algorithms used in the backward phase of our Gb. As in the generic IT garbling, IT* uses ITAND, ITXOR, and BwNOT algorithms, and is applicable to circuits with only TypeS wires. Additionally, it allows the adversary to choose a set of output wires, which will have the same offset, i.e., the XOR of the keys for each of those wires is fixed.

Lemma 2. *The IT* garbling scheme is information-theoretically authentic.*

Proof. Any circuit f with only TypeS wires will be a set of independent subcircuits $\{f_1, \ldots, f_n\}$, each having a 1-bit output unconnected from each other. Each subcircuit f_i that the adversary does not choose to have the same offset is garbled independently, which is proven to be information-theoretically authentic by [24]. Hence, we will consider only the set of circuits $\{f'_1, \ldots, f'_m\}$ that the adversary chooses to have the same Δ. Also, via the reduction from [24], the authenticity of each of those f'_i can be reduced to the authenticity of a one-gate circuit, i.e., its output gate's authenticity. Hence, we assume each of the circuits $f'_1, \ldots, f'_m$ has only one gate (it may be any of AND or XOR or NOT) and have the same Δ at the output. We highlight that the IT* garbling scheme still has size-zero, i.e., the adversary receives only $\hat{X}$ (specifically not $\hat{F}$) upon garbling. Therefore, the information-theoretic authenticity can be proven by proving the same $\hat{X}$ may be generated with the same probability independent of Δ for any given circuit $f = \{f'_1, \ldots, f'_m\}$ where each f'_i is a gate and its input $\hat{x}$ whose bits are TypeS. We show this gate-by-gate for each of AND, XOR, and NOT gates.

AND gate. Given a gate i, with input wires a and b, the garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, sets $W_i^1 \leftarrow W_i^0 \oplus \Delta$ and then calls GbITAND to obtain the wire keys W_a^0, W_a^1, W_b^0, and W_b^1. Then, either W_a^0 or W_a^1 is sent as the wire key X_a and either W_b^0 or W_b^1 is sent as the wire key X_b, depending on the values of w_a and w_b. We will compare for all combinations of w_a and w_b, the probability of obtaining the same keys X_a and X_b using Δ_0 with the one using Δ_1.

Case $w_a = 0$ and $w_b = 0$. The garbling algorithm picks W_i^0 uniformly random, and sets $(W_a^0, W_b^0) \leftarrow (W_i^0, W_i^0)$ via GbITAND. Hence, any given pair of $X_a = W_a^0$ and $X_b = W_b^0$ (the given two keys for $w_a = 0$ and $w_b = 0$) could be generated with equal probability using Δ_0 or Δ_1.

Case $w_a = 1$ *and* $w_b = 0$. The garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random and sets $W_b^0 \leftarrow W_i^0$ via $\mathsf{GbITAND}$. $\mathsf{GbITAND}$ also picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$. Again, due to directly picking uniformly random, any given pair of $X_a = W_a^1$ and $X_b = W_b^0$ could be generated with equal probability using either Δ_0 or Δ_1.

Case $w_a = 0$ *and* $w_b = 1$. The garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, and sets $W_a^0 \leftarrow W_i^0$ via $\mathsf{GbITAND}$. $\mathsf{GbITAND}$ also picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, and sets $W_b^1 \leftarrow W_a^1 \oplus W_i^1$ where $W_i^1 = W_i^0 \oplus \Delta$. Any given pair of X_a and X_b could be generated with equal probability using either Δ_0 or Δ_1 due to the following reasons. Due to directly picking W_i^0 uniformly random, $X_a = W_a^0 = W_i^0$ could be generated with equal probabilities using either Δ_0 or Δ_1. Also, W_b^1 is generated by XORing the uniformly picked W_a^1 with some value W_i^1. Any element of $\{0,1\}^\ell$ could be generated as W_b^1 with equal probability regardless of W_i^1's value, and then set as $X_b = W_b^1$.

Case $w_a = 1$ *and* $w_b = 1$. The garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, and sets $W_i^1 \leftarrow W_i^0 \oplus \Delta$, hence W_i^1 could get any value from $\{0,1\}^\ell$ with equal probability using either Δ_0 or Δ_1. $\mathsf{GbITAND}$ then picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, and sets $W_b^1 \leftarrow W_a^1 \oplus W_i^1$. We deduce $W_b^1 = W_a^1 \oplus W_i^0 \oplus \Delta$ Any given pair of X_a and X_b could be generated with equal probability using either Δ_0 or Δ_1 since X_a directly comes from uniform picking of W_a^1 and X_b results from XORing some values with uniform picked W_i^0.

W_a^0 and W_b^1 are prepared independently, so their probabilities can be considered separately. Due to directly choosing W_a^0 uniformly, X_a^0 could be generated with equal probabilities using either Δ_0 or Δ_1. Due to the generation of W_b^1 by XORing the uniformly randomly picked W_a^1 with some value W_i^1, regardless of $W_i^1 = W_i^0 \oplus \Delta_0$ or $W_i^1 = W_i^0 \oplus \Delta_1$, any element of $\{0,1\}^\ell$ could be generated as W_b^1 with equal probability.

XOR gate. Similar to AND gate garbling, given a gate i, with input wires a and b, the garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, sets $W_i^1 \leftarrow W_i^0 \oplus \Delta$ and then calls GbXOR to obtain W_a^0, W_a^1, W_b^0, and W_b^1. Then, either W_a^0 or W_a^1 is sent as X_a and either W_b^0 or W_b^1 is sent as X_b, depending on the values of w_a and w_b. We will compare for all combinations of w_a and w_b, the probability of obtaining the same keys X_a and X_b using Δ_0 with one using Δ_1.

Case $w_a = 1$ *and* $w_b = 1$. The garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random. $\mathsf{GbITXOR}$ then picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$, and sets $W_b^1 \leftarrow W_a^1 \oplus W_i^0$. As W_a^1 is picked uniformly and W_b^1 is randomized by independent choice of W_i^0, any given pair of $X_a = W_a^1$ and $X_b = W_b^1$ could be generated with equal probability using either Δ_0 or Δ_1.

Case $w_a = 1$ *and* $w_b = 0$. The garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, and sets $W_i^1 \leftarrow W_i^0 \oplus \Delta$. $\mathsf{GbITXOR}$ then picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$, and sets $W_b^0 \leftarrow W_a^1 \oplus W_i^1$, which means $W_b^0 = W_a^1 \oplus W_i^0 \oplus \Delta$. As W_a^1 is picked uniformly random, and W_b^0 is randomized by independent choice of W_i^0, any given pair of $X_a = W_a^1$ and $X_b = W_b^0$ could be generated with equal probability using Δ_0 or Δ_1.

Case $w_a = 0$ *and* $w_b = 1$. The garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$ uniformly random, and sets $W_i^1 \leftarrow W_i^0 \oplus \Delta$. GbITXOR then picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$, and sets $W_b^1 \leftarrow W_a^1 \oplus W_i^0$. It also sets $W_a^0 \leftarrow W_b^1 \oplus W_i^1$, which means $W_a^0 = W_b^1 \oplus W_i^0 \oplus \Delta$. As W_b^1 is randomized by W_a^1, and W_a^0 is additionally randomized by the choice of W_i^0, any given pair of $X_a = W_a^0$ and $X_b = W_b^0$ could be generated with equal probability using either Δ_0 or Δ_1.

Case $w_a = 0$ *and* $w_b = 0$. The garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$. GbITXOR then picks $W_a^1 \twoheadleftarrow \{0,1\}^\ell$. Via XOR arithmetic, it is easy to show that, upon execution of GbITXOR, $W_a^0 = W_a^1 \oplus \Delta$ and $W_b^0 = W_a^1 \oplus W_i^0 \oplus \Delta$. As W_b^0 is randomized by W_a^1, and W_a^0 is additionally randomized by the choice of W_i^0, any given pair of $X_a = W_a^0$ and $X_b = W_b^0$ could be generated with equal probability using either Δ_0 or Δ_1.

NOT gate. Given a gate i with an input wire a, the garbling algorithm picks $W_i^0 \twoheadleftarrow \{0,1\}^\ell$, sets $W_i^1 \leftarrow W_i^0 \oplus \Delta$, and then swaps the order of the generated keys to obtain W_a^0 and W_a^1. As W_a^1 is set via directly picking W_i^0 uniformly random, in case of $w_a = 1$, using either Δ_0 or Δ_1, X_a could be obtained with equal probability. Also, as W_a^0 is randomized by W_i^0, in case of $w_a = 0$, again using either Δ_0 or Δ_1, X_a could be obtained with equal probability.

We conclude that each of the gates $f'_1, \ldots, f'_n$ is garbled independently with independently picked randomizations, and the shown equality of probabilities that X_a and X_b is obtained from Δ_0 and Δ_1 extends to the whole set $\hat{X}$. □

Theorem 4. *The AuthOr garbling scheme satisfies verifiability if the hash function H has circular correlation robustness (CCR).*

Proof (Intuition). Our verifiability algorithm VE calls gate verification algorithms from Table 2 in topological order based on the gate garbling schemes used. Each gate garbling algorithm (except for VeNOT) ensures the output wire key has only two possible assignments (unless the hash function H is shown to be insecure), one for truth value 0 and one for truth value 1. VeXOR does so by ensuring the input wire offsets are the same, which means $W_i^0 = W_a^0 \oplus W_b^0 = W_a^1 \oplus W_b^1$ and $W_i^1 = W_a^0 \oplus W_b^1 = W_a^1 \oplus W_b^0$. The verification algorithms for HG garbling also guarantee the input wire offsets Δ are the same, in addition to that $W_i^0 = H(W_a^0) = W_b^0 \oplus H(i, W_a^1)$ holds (F_i is added for HG2). As the inputs have the same offset Δ, the XOR arithmetic ensures $W_i^1 = W_i^0 \oplus \Delta$. We have inherited VeITAND from [24], which has shown that the IT garbling scheme is verifiable. Also, all of the given gate verification algorithms return the two output wire keys of each gate till the end of the circuit. Hence, for output gates, there exist only two possible assignments. Regarding the first probability of verifiability, this ensures that the obtained wire keys will be equal, and this probability is always 0. Regarding the second probability of verifiability, we can modify Ve to build an Ext. Upon obtaining all the output wire keys using Ve, Ext finds the corresponding output keys to the bits of y and returns them. Again, this ensures the obtained wire keys will be equal to the ones that come from evaluation, and this probability is always 0. □

Circuit	[37]			**AuthOr**			
	Gb cost	Ev cost	GC Size	Gb cost	Ev Cost	GC Size	Size Gain (%)
adder64	0.003	0.001	64	0.010	0.001	64	0.00
sub64	0.002	0.001	64	0.003	0.001	64	0.00
mult64	0.111	0.041	4034	0.116	0.042	3970	1.59
mult2-64	0.224	0.085	8129	0.234	0.087	4034	**50.38**
divide64	0.158	0.068	4665	0.169	0.070	4664	0.02
udivide64	0.123	0.049	4286	0.128	0.051	4225	1.42
and8	0.000	0.000	8	0.000	0.000	1	**87.50**
zero-equal	0.002	0.000	64	0.000	0.000	1	**98.44**
FP-add	0.140	0.049	5369	0.147	0.051	5348	0.39
FP-floor	0.016	0.006	645	0.017	0.006	644	0.16
FP-ceil	0.016	0.006	645	0.017	0.006	645	0.00
FP-div	2.090	0.746	80184	2.179	0.764	80163	0.03
FP-sqrt	2.350	0.844	89964	2.455	0.857	89964	0.00
FP-f2i	0.038	0.013	1464	0.042	0.014	1456	0.55
FP-mul	0.501	0.176	19426	0.523	0.180	19406	0.10
FP-eq	0.009	0.003	316	0.009	0.003	305	3.48
FP-i2f	0.064	0.022	2407	0.068	0.023	2407	0.00
cmp32-unsigned-lt	0.004	0.001	151	0.004	0.001	141	**6.62**
cmp32-signed-lteq	0.004	0.001	151	0.004	0.001	138	**8.61**
							Average: 13.67

Table 6: Experiment results on arithmetic circuits of [4]. Computation costs and GC sizes are in terms of seconds and the number of generated ciphertexts, respectively.

6 Implementation and Experimental Results

Our implementations of AuthOr as proof-of-concept and the state-of-the-art HG garbling [37] for comparison, both in Python 3 are given at [2]. We have run tests on the arithmetic operation circuits of [4], the cryptographic circuits of [4], and benchmark circuits of [20], all of which are constructed by independent researchers from us. For each circuit, we have repeated our tests 10 times on an Apple M2 MacBook Pro with 24 GB of RAM. We present the average experimental results in Table 6, 7, and Table 8, which compare costs for garbling and evaluating computation times (measured as elapsed times in seconds) and the GC size (measured as the total number of ciphertexts generated). As the bottleneck of garbling schemes is GC size [37], we specifically calculate the GC Size Gain of our scheme in the last columns of the given tables. We did not measure the overhead associated with the compilation and preprocessing steps required before garbling.

Preprocessing of the circuit. In our implementation, we developed a compiler to parse circuits written in both supported syntaxes and to manage this emulation process. [4] implemented a buffer via an AND gate with both input pins connected to the same wire for some circuits. Since these gates do not affect

Circuit	[37]			**AuthOr**			
	Gb cost	Ev cost	GC Size	Gb cost	Ev Cost	GC Size	Size Gain (%)
sha256	0.767	0.325	22574	0.823	0.334	22569	0.02
aes128	0.210	0.087	6401	0.227	0.090	6401	0.00
aes192	0.238	0.098	7169	0.256	0.103	7169	0.00
Keccak-f	1.211	0.474	38401	1.299	0.493	38401	0.00
sha512	1.973	0.836	57948	2.133	0.857	57943	0.01
aes256	0.292	0.121	8833	0.313	0.126	8833	0.00
							Average: 0.01

Table 7: Experimental results on cryptographic circuits of [4]. Computation costs and GC sizes are in terms of seconds and the number of generated ciphertexts, respectively.

Circuit	[37]			**AuthOr**			
	Gb cost	Ev cost	GC Size	Gb cost	Ev Cost	GC Size	Size Gain (%)
c5315	0.055	0.016	2012	0.044	0.014	1229	**38.92**
c1908	0.017	0.005	576	0.014	0.004	398	**30.90**
c432	0.004	0.001	156	0.003	0.001	83	**46.79**
c6288	0.065	0.020	2385	0.056	0.017	1650	**30.82**
c7552	0.070	0.021	2491	0.060	0.019	1604	**35.61**
c17	0.000	0.000	7	0.000	0.000	1	**85.71**
c2670	0.024	0.007	876	0.019	0.006	514	**41.32**
c499	0.003	0.001	91	0.002	0.001	49	**46.15**
c880	0.009	0.003	331	0.007	0.002	171	**48.34**
c1355	0.014	0.004	507	0.010	0.003	213	**57.99**
c3540	0.033	0.010	1169	0.029	0.009	784	**32.93**
							Average: 45.05

Table 8: Experimental results on benchmark circuits of [20]. Computation costs and GC sizes are in terms of seconds and the number of generated ciphertexts, respectively.

the circuit's mathematical functionality or the authenticity of garbling, we remove them in a preprocessing step. Additionally, some benchmarks include pins that are set as 0 through an XOR gate with a common input. For similar reasons, we also removed these XOR gates. Additionally, we employed the "networkx" library to determine the topological order of the gates in each Boolean circuit. Our implementation supports two syntaxes for representing boolean circuits: *Bristol Fashion* [4] and *Bench*. In both formats, each circuit is represented by listing all internal Boolean gates and their pin connectivity. The set of gates discussed in Section 3 is Turing complete, allowing us to emulate all other types of gates, including those with a fan-in greater than 2, as well as OR, NOR, NAND, and XNOR gates.

Garbling. The types of wires can be determined during the forward phase, which our implementation takes advantage of. We made this separate in Table

5 only for the sake of clarity. As suggested by [6,37], the hash function H can be implemented by using a symmetric key encryption algorithm with a shared fixed key k. We use AES128 encryption in CBC mode (inherited from the "Cryptography" library) to set the hash function H as:

$$H(i, W) = AES128_k(W) \oplus i \oplus W$$

GC size comparison with HG garbling [37]. As shown in Tables 6, 7, and 8, the GC size gain is highly dependent on the architecture of the circuits. For cryptographic circuits of [4], the gain is marginal because the complexity of gate connections requires our scheme to garble almost all the gates with GaHG2 during the forward phase. In contrast, for arithmetic circuits of [4] and for benchmark circuits of [4], the gain is up to roughly 98% and 86%, and averages at roughly 14% and 45%, respectively.

References

1. S. Agrawal. Stronger security for reusable garbled circuits, general definitions and attacks. In *CRYPTO '17*, 2017.
2. A. Ajorian. Implementation of author and half gates garbling schemes. Accessible via `https://github.com/Ajorian/AuthOr/tree/main#`.
3. P. Ananth and A. Lombardi. Succinct garbling schemes from functional encryption through a local simulation paradigm. In *TCC '18*, 2018.
4. D. Archer, V. Arribas Abril, S. Lu, P. Maene, N. Mertens, D. Sijacic, and N. Smart. 'Bristol Fashion' MPC Circuits. Retrieved 2025-04-10 from `https://nigelsmart.github.io/MPC-Circuits/`.
5. S. Basu and L. Parida. Quantum analog of shannon's lower bound theorem, 2023.
6. M. Bellare, T. Hoang, S. Keelveedhi, and P. Rogaway. Efficient garbling from a fixed-key blockcipher. In *IEEE SP '13*, 2013.
7. Mihir Bellare, Viet Tung Hoang, and Phillip Rogaway. Foundations of garbled circuits. In *ACM CCS '12*, 2012.
8. D. Boneh, C. Gentry, S. Gorbunov, S. Halevi, V. Nikolaenko, G. Segev, V. Vaikuntanathan, and D. Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit abe and compact garbled circuits. In *EUROCRYPT '14*, 2014.
9. R. Canetti, J. Holmgren, A. Jain, and V. Vaikuntanathan. Succinct garbling and indistinguishability obfuscation for ram programs. In *ACM STOC '15*, 2015.
10. M. Chase, C. Ganesh, and P. Mohassel. Efficient zero-knowledge proof of algebraic and non-algebraic statements with applications to privacy preserving credentials. In *CRYPTO '16*, 2016.
11. S. G. Choi, J. Katz, R. Kumaresan, and H.-S. Zhou. On the security of the "free-xor" technique. In *TCC '12*, 2012.
12. N. Döttling and S. Garg. Identity-based encryption from the diffie-hellman assumption. In *CRYPTO '17*, 2017.
13. T. K. Frederiksen, J. B. Nielsen, and C. Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In *EUROCRYPT '15*, 2015.
14. C. Ganesh, Y. Kondi, A. Patra, and P. Sarkar. Efficient adaptively secure zero-knowledge from garbled circuits. In *PKC '18*, 2018.
15. R. Gennaro, C. Gentry, and B. Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In *CRYPTO '10*, 2010.

16. R. Gennaro, C. Gentry, B. Parno, and M. Raykova. Quadratic span programs and succinct nizks without pcps. In *EUROCRYPT '13*, 2013.
17. S. Goldwasser, Y. Kalai, R. A. Popa, V. Vaikuntanathan, and N. Zeldovich. Reusable garbled circuits and succinct functional encryption. In *ACM STOC '2013*.
18. S. Gueron, Y. Lindell, A. Nof, and B. Pinkas. Fast garbling of circuits under standard assumptions. In *ACM CCS '15*, 2015.
19. M. Jawurek, F. Kerschbaum, and C. Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In *ACM CCS '13*, 2013.
20. M. Jenihhin. Bench marks. Retrieved 2025-04-10 from `https://pld.ttu.ee/~maksim/benchmarks/iscas85/bench/`.
21. C. Kempka, R. Kikuchi, S. Kiyoshima, and K. Suzuki. Garbling scheme for formulas with constant size of garbled gates. In *ASIACRYPT '15*, 2015.
22. V. Kolesnikov. Gate evaluation secret sharing and secure one-round two-party computation. In *ASIACRYPT '05*, 2005.
23. V. Kolesnikov and T. Schneider. Improved garbled circuit: Free xor gates and applications. In *ICALP '08*, 2008.
24. Y. Kondi and A. Patra. Privacy-free garbled circuits for formulas: Size zero and information-theoretic. In *CRYPTO '17*, 2017.
25. Y. Lindell and B. Pinkas. An efficient protocol for secure two-party computation in the presence of malicious adversaries. In *EUROCRYPT '07*, 2007.
26. Y. Lindell and B. Pinkas. Secure two-party computation via cut-and-choose oblivious transfer. In *TCC '11*, 2011.
27. Y. Lindell, B. Pinkas, N. Smart, and A. Yanai. Efficient constant round multi-party computation combining bmr and spdz. In *CRYPTO '15*, 2015.
28. Y. Lindell and B. Riva. Blazing fast 2pc in the offline/online setting with security for malicious adversaries. In *ACM CCS '15*, 2015.
29. H. Liu, X. Wang, K. Yang, and Y. Yu. Garbled circuits with 1 bit per gate. In *Cryptology ePrint Archive, Paper 2024/1988*, 2024.
30. G. Marcadet, P. Lafourcade, and L. Robert. Rmc-pvc: A multi-client reusable verifiable computation protocol. In *ACM SAC '23*, 2023.
31. R. Nieminen and T. Schneider. Breaking and fixing garbled circuits when a gate has duplicate input wires. *Journal of Cryptology*, 36, 08 2023.
32. M. Rosulek and L. Roy. Three halves make a whole? beating the half-gates lower bound for garbled circuits. In *CRYPTO '21*, 2021.
33. C. E. Shannon. The synthesis of two-terminal switching circuits. *The Bell System Technical Journal*, 28(1):59–98, 1949.
34. X. Wang, S. Ranellucci, and J. Katz. Global-scale secure multiparty computation. In *ACM CCS '17*, 2017.
35. A. C. Yao. Protocols for Secure Computations. In *IEEE FOCS '82*, 1982.
36. A. C. Yao. How to generate and exchange secrets. In *IEEE FOCS '86*, 1986.
37. S. Zahur, M. Rosulek, and D. Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. In *EUROCRYPT '15*, 2015.

A Inefficiency of IT Garbling for Generic Circuits

For fairness of comparison with IT garbling [24], we convert an example generic circuit (with size g) given in Figure 3 into a boolean formula by applying the technique mentioned in [24] to convert a circuit into a boolean formula, when an input wire i of the circuit is inputted to n gates by replacing i by n wires.

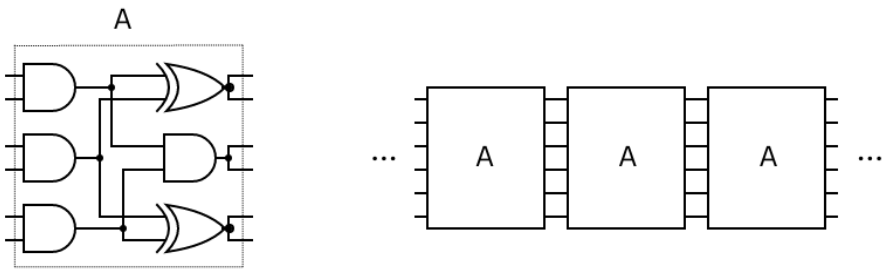


Fig. 3: An example circuit with size g obtained by repeating A-boxes $g/6$ times. A-box construction is also shown. Circuits are left to right in topological order.

Note that each circuit layer has 3 gates, so the depth of the circuit is $g/3$. Starting from the last layer in inverse topological order, the procedure replaces each fan-out-n gate i with n fan-out-1 gates, each of which has the same input wires as the gate i had. The last layer $(g/3)$-th remains the same as it does not have any fan-out-2 gates. The $(g/3-1)$-th layer has 3 fan-out-2 gates, so is replaced with 6 gates. However, now the $(g/3-2)$-th layer has 3 fan-out-4 gates, so it needs to be replaced by 12 gates. At each lay the number of the gates doubles, hence we calculate the size g' of the obtained boolean formula as $g' = \sum_{j=0}^{g/3-1} 3 \cdot 2^j = 3 \cdot 2^{g/3} - 3$, which means $g' \in \Theta(exp(g))$.